

1. What You MUST remember

Session 1 - basics: variables, data types, input

Session 3 - Math, spread, conditions

Session 5 - DOM updates, arrays, iteration

Session 2 - operators, logic, ternary

Session 4 - strings, numbers, DOM

VARIABLES

var: redeclare + reassign allowed.

let: reassign allowed; same-scope redeclaration not allowed.

const: must be initialized immediately; binding cannot be reassigned.

DECLARATION vs INITIALIZATION

Declaration = create the variable name.

Initialization = give it the first value.

Reassignment = change an existing value.

Redeclaration = declare the same name again.

DATA TYPES

Primitive: String, Number, Boolean, Undefined, Null, BigInt.

Reference / object types: Array, Object.

Array = ordered values, index starts at 0.

Object = key-value data; use `obj.key` or `obj["key"]`.

20-MINUTE MEMORY LINE

choose variable -> know type -> know whether it can change -> remember index 0.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

2. INPUT -> CONVERT -> OUTPUT

INPUT

`prompt()` collects browser input as text (or `null` if cancelled).

Never assume "10" is numeric 10.

NUMBER()

Use `Number()` when text should become a JavaScript number:

`Number("10") -> 10`

Invalid numeric text can become `NaN`.

TYPE CHECK

`typeof` reports a type string.

`typeof "Hi" -> "string"`

`typeof 56 -> "number"`

`typeof true -> "boolean"`

`typeof 123n -> "bigint"`

`typeof [] / {} -> "object"`

`typeof null -> "object"` (historical quirk)

CLASSIC PITFALL

`"10" + "20" -> "1020"`

`Number("10") + Number("20") -> 30`

```
let a = Number(prompt("Enter Num1:"));
let b = Number(prompt("Enter Num2:"));
console.log("Addition:", a + b);
console.log("Type:", typeof (a + b));
```

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

3. LAST-MINUTE CHECK - SESSION 1

COMMON TRAPS

const needs an initializer. / prompt gives text. / array first index is 0. / null != undefined. / redeclaration != reassignment.

RAPID QUESTIONS

1. Third array item index? **2**.
2. `typeof 123n`? **"bigint"**.
3. Why `Number(prompt())`? **numeric arithmetic**.
4. Undefined vs null? **unassigned vs intentional absence**.

MINI PROGRAM

Ask two numbers, convert with `Number()`, add them, print result and `typeof`. This combines the whole session.

MUST-SAY OUT LOUD

"I know what a variable is, I know which declaration keyword to choose, and I know input is text until I convert it."

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

1. OPERATORS - THE CORE SET

VOCABULARY

Operator performs an operation. **Operand** is a value used by it. **Expression** evaluates to a value.

ARITHMETIC

`+` `-` `*` `/` `%` `**`

`%` = remainder. `**` = exponentiation.

INCREMENT / DECREMENT

`++` `--` change a variable by 1.

`++x` = change first, use new value.

`x++` = use current value, then change.

Same timing rule for `--x` and `x--`.

ASSIGNMENT

`=` `+=` `-=` `*=` `/=`

`x += y` means `x = x + y`.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

2. COMPARISON, LOGIC & TERNARY

COMPARISON

`==` loose equality (coercion may happen).
`===` strict equality (value + type).
`!=` loose inequality.
`!==` strict inequality.
`>` `<` `>=` `<=` directional comparisons.

LOGICAL

`&&` AND -> both true.
`||` OR -> at least one true.
`!` NOT -> flips boolean.

SHORT-CIRCUIT

`false && anything` -> result known already.
`true || anything` -> result known already.

TERNARY

`condition ? valueIfTrue : valueIfFalse`
Best for a short two-way value choice.

```
let age = 18;  
const msg = age >= 18 ? "Eligible" : "Not eligible";  
  
21 % 2 === 0    // false -> odd
```

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

3. 20-MINUTE CRAM - SESSION 2

FAST RECIPES

Even: `n % 2 === 0`

Positive: `n >= 0`

Larger of 2: `a > b ? a : b`

Both conditions: `cond1 && cond2`

Either condition: `cond1 || cond2`

TRACE RULE

For ++/--, write two columns in your head: **value produced** and **variable after**.

INTERVIEW LINES

`%` is remainder.

`+=` updates the same variable.

`===` is strict equality.

Ternary is an expression that produces a value.

SELF-TEST

Can you predict `let x=5; console.log(x++); console.log(++x); console.log(x);`? -> 5, 7, 7.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

1. MATH OBJECT - READY-MADE TOOLS

METHODS

`Math.random()` -> 0 inclusive, 1 exclusive.

`Math.floor(x)` -> rounds down.

`Math.sqrt(x)` -> square root.

`Math.pow(a,b)` -> a to the b power.

`Math.min(...)` -> smallest.

`Math.max(...)` -> largest.

RANDOM INTEGER

`Math.floor(Math.random() * 10)` -> 0..9.

For 1..100: `Math.floor(Math.random() * 100) + 1`.

MIN / MAX WITH ARRAYS

`Math.min/max` expect separate arguments. When values are in an array, use spread: `Math.max(...numbers)`.

POWER NOTE

`Math.pow(2,5)` and `2 ** 5` both produce 32.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

2. SPREAD + CONDITIONAL CONTROL FLOW

SPREAD (...)

Unpacks an iterable into individual items.

```
const n=[10,20,30]; Math.max(...n)
```

Read it as: "take the items out one by one here."

IF / ELSE IF / ELSE

Runs the first matching branch.

```
if (condition) { ... }  
else if (other) { ... }  
else { ... }
```

TERNARY

```
condition ? trueValue : falseValue
```

Compact two-way decision; the result can be stored in a variable.

FLOW

condition -> choose branch -> execute that block -> skip later branches once a match is found.

```
const nums = [10,20,30];  
console.log(Math.max(...nums));  
  
let grade;  
if (score >= 90) grade = "A+";  
else if (score >= 70) grade = "A";  
else grade = "B";
```

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

3. 20-MINUTE CRAM - SESSION 3

PROBLEM PATTERNS

Voting: validate first, then check 18+.
Largest 2: ternary.
Largest 3+: Math.max.
Divisible by both: `x%a===0 && x%b===0`.
Range: `x>=min && x<=max`.
Positive/even: `x>0 && x%2===0`.

ACCESS / LOGIN

Two credentials -> compare both with `===` and combine with `&&`.

GRADING / TEMPERATURE

Use ordered `if / else if` thresholds. Put the stricter / higher threshold first.

SELF-TEST

Can you build: **PIN check** -> **withdrawal amount valid** -> **balance sufficient** -> **success**? If yes, Session 3 is solid.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

1. STRING METHODS - FAMILY MAP

INSPECT

`length` = size; `charAt/at` = character access.

SEARCH

`indexOf` / `lastIndexOf` = first / last index. `includes` = yes/no. `search` / `match` = regex-oriented search.

EXTRACT

`slice(start,end)` -> end excluded; negatives count from end. `substring` = index extraction with its own index rules.

TRANSFORM / TEST

`toLowerCase`, `toUpperCase`, `replace`, `replaceAll`, `concat`, `startsWith`, `endsWith`, `isWellFormed`.
Most String methods return a new string; they do not rewrite the original primitive.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

2. NUMBER CHECKING + CONVERSION

CHECKING

`isNaN(x)` = global check after numeric conversion.
`Number.isNaN(x)` = actual NaN only.
`Number.isFinite(x)` = finite number.
`Number.isInteger(x)` = integer.
`Number.isSafeInteger(x)` = safe integer.

CONVERSION

`Number(x)` = whole-value conversion.
`parseInt(x, 10)` = integer prefix.
`parseFloat(x)` = decimal prefix.

CLASSIC EXAMPLE

`Number("12.34abc")` -> NaN
`parseInt("12.34abc", 10)` -> 12
`parseFloat("12.34abc")` -> 12.34

MEMORY LINE

Number = whole value | parseInt = integer prefix |
parseFloat = decimal prefix

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

3. DOM FUNDAMENTALS - SELECT -> READ

CORE SELECTORS

`getElementById()` -> one id element.
`getElementsByTagName()` -> HTMLCollection.
`querySelector()` -> first CSS match.
`querySelectorAll()` -> NodeList of all matches.
`getElementsByClassName()` -> class collection.

READ COMMON VALUES

Inputs: `.value`.
Visible text: `.textContent`.

HTML -> JS FLOW

input id -> button click -> function -> select element -> read property.

TRAP

IDs should be unique. Duplicate ids can make id-based selection misleading.

```
function getData() {  
  const el = document.getElementById("name");  
  console.log(el.value);  
  const ps = document.getElementsByTagName("p");  
  console.log(ps[0].textContent);  
}
```

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

1. DOM UPDATE TOOLKIT

SELECT

`getElementById()` -> start by finding the element.

CONTENT + STYLE

`innerHTML` inserts/replaces HTML markup. Use carefully with untrusted input.

`element.style.color`, `backgroundColor`, `fontSize`, `borderLeft` etc. CSS properties become camelCase in JS.

ATTRIBUTES

`setAttribute(name,value)` set/change.

`getAttribute(name)` read (or null).

`hasAttribute(name)` boolean.

`removeAttribute(name)` delete.

EVENTS

`onclick="change()"` runs the function after a click. Later, `addEventListener("click", handler)` keeps behavior separate from HTML.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

2. ARRAYS - MUTATE, REARRANGE, ACCESS

END / FRONT

`push()` add end -> returns new length.
`pop()` remove end -> returns removed item.
`shift()` remove front -> returns removed item.
`unshift()` add front -> returns new length.

SPLICE

`splice(index,0,value)` add.
`splice(index,count)` delete.
`splice(index,count,value)` replace.
It mutates the original and returns removed items.

REARRANGE

`join("_")` -> string, no mutation.
`reverse()` -> mutates.
`sort((a,b)=>a-b)` -> numeric ascending, mutates.

INDEXING

Last index = `arr.length - 1`.
Second-last = `arr.length - 2`.

```
const arr = [11,22,33,44];
arr.splice(1,0,99);
console.log(arr);
console.log(arr[arr.length - 2]);
```

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.

3. SEARCH + ITERATION + FINAL SELF-TEST

NON-MUTATING

`slice` copies a portion. `concat` combines into a new array. These do not change the original.

SEARCH

`includes` -> exists? `indexOf` -> first index. `lastIndexOf` -> last index.

ITERATION

`forEach` -> run code for each item, no new array.
`map` -> transform to new array.
`filter` -> keep matches.
`find` / `findIndex` -> first matching value / index.
`some` / `every` -> at least one / all pass.

MASTER MEMORY MAP

DOM: `select` -> `read/change` -> `attribute/event`.

Arrays: `add/remove` -> `insert/delete/replace` -> `join/reverse/sort` -> `search` -> `process`.

20-minute route: 7 min read the cards -> 7 min rewrite the code example from memory -> 6 min answer the self-test without looking.